Sky: Spark Vaults v2

Security Review

Cantina Managed review by:

Christoph Michel, Lead Security Researcher
Mario.eth, Lead Security Researcher

October 1, 2025

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**

3 Findings **4**
3.1 Low Risk . . . 4
3.1.1 `totalSupply` can overflow in `mint` . . . 4
3.1.2 `previewRedeem` and `previewWithdraw` revert when vault is missing liquidity . . . 4
3.2 Informational . . . 5
3.2.1 Missing functions from `ISparkVault` . . . 5
3.2.2 Taker can `take` assets out of vault and `deposit` them . . . 5

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Sky Protocol is a decentralised protocol developed around the USDS stablecoin.

From Aug 26th to Sep 1st the Cantina team conducted a review of spark-vaults-v2 on commit hash 8f1442ad. The team identified a total of **4** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	2	1	1
Gas Optimizations	0	0	0
Informational	2	2	0
Total	**4**	**3**	**1**

The Cantina Managed team reviewed Sky's `spark-vaults-v2`'s holistically on commit hash 0a686ba2 and concluded that all findings were addressed and no new vulnerabilities were identified.

3 Findings

3.1 Low Risk

3.1.1 `totalSupply` can overflow in `mint`

Severity: Low Risk

Context: SparkVault.sol#L428-L433

Description: When depositing assets, the user is minted `shares` at the current share price `nowChi()`. The `_mint` function performs an **unchecked** increase in `totalSupply`:

```
function _mint(uint256 assets, uint256 shares, address receiver) internal {
    require(receiver != address(0) && receiver != address(this), "SparkVault/invalid-address");

    _pullAsset(msg.sender, assets);

    // NOTE: Don't need overflow checks as balanceOf[receiver] <= totalSupply
    //       and shares <= totalSupply
    unchecked {
        balanceOf[receiver] = balanceOf[receiver] + shares;
        totalSupply = totalSupply + shares;
    }
    // ...
}
```

However, the comment does not correctly explain why this unchecked addition is safe for the `totalSupply`. In fact, it is incorrect and can overflow if the total deposited assets are more than `uint256.max`. This is possible for this vault as the `taker` can always take out the funds:

1. `taker` takes out entire vault assets.
2. `taker` deposits these vault assets and is minted new shares.
3. Repeat.

When the `totalSupply` overflows, the `totalAssets()` will reflect this and be significantly reduced. Low severity is given as this overflow requires a privileged, trusted role (`taker`) to misbehave.

Recommendation: Consider performing the `totalSupply = totalSupply + shares;` first, outside the `unchecked` block. Then add a comment that the `balanceOf` unchecked addition is secure "as `balanceOf[receiver] <= totalSupply`".

Sky: Fixed in PR 36.

Cantina Managed: Fix verified.

3.1.2 `previewRedeem` and `previewWithdraw` revert when vault is missing liquidity

Severity: Low Risk

Context: SparkVault.sol#L349-L363

Description: EIP-4626 says the following about whether the `preview*` functions should revert:

> MUST NOT account for redemption limits like those returned from maxRedeem and should always act as though the redemption would be accepted, regardless if the user has enough shares, etc... MUST NOT revert due to vault specific user/global limits. MAY revert due to other conditions that would also cause redeem to revert.

However, the `previewRedeem` and `previewWithdraw` revert when the vault is missing liquidity:

```
function previewRedeem(uint256 shares) external view returns (uint256 amount) {
    amount = convertToAssets(shares);
    require(
        IERC20(asset).balanceOf(address(this)) >= amount,
        "SparkVault/insufficient-liquidity"
    );
}
```

This check could be interpreted as a ”global limit” according to the spec and should therefore not revert.

Recommendation: Consider removing the liquidity checks in the `preview*` functions (but keep them in the `max*` functions). If there's a reason to diverge from the 4626-spec, consider documenting this behavior.

Sky: Acknowledged. We read this spec and we interpreted a global limit as something like a supply cap. A lack of liquidity is more of a condition of the vault based on the current state so it would be relevant to revert in this case as per

> MAY revert due to other conditions that would also cause redeem to revert.

Cantina Managed: Acknowledged.

3.2 Informational

3.2.1 Missing functions from `ISparkVault`

Severity: Informational

Context: ISparkVault.sol

Description: The following functions are missing from the `ISparkVault`:

- `nowChi`.
- `setVsrBounds`.
- Whole `IAccessControlEnumerable` interface.

Recommendation: Consider adding these additional functions to the interface

Sky: Fixed in PR 33.

Cantina Managed: Fix verified.

3.2.2 Taker can `take` assets out of vault and `deposit` them

Severity: Informational

Context: SparkVault.sol#L133

Description: The taker can `take` out the entire deposited assets from the vault. This is done so they can earn yield on it which is then later transferred back to the vault. However, a taker can take out this amount and deposit it again to receive shares. This can be repeated ad infinitum, each time `totalAssets()` is inflated by the deposited amount. `totalAssets()` will track an inflated value as the `taken` assets are double-counted and are unlikely to be paid back.

Recommendation: As the taker already needs to be fully trusted because they can `take` out arbitrary `asset` amounts out of the vault, this attack is within the existing security model, and no further actions need to be taken. Attention must be paid to the possible actions a taker can perform if this role is given to a potentially untrusted smart contract (like the ALM proxy controlled by a relayer, only restricted by the controller code).

Sky: Addressed in PR 34.

Cantina Managed: Mitigated. This has been addressed by disabling `deposit/mint` for the `TAKER` role. Note that a `TAKER` could still transfer the tokens to another wallet under their control and deposit from this wallet.